

Hadoop Infrastructure Explained

With a Practical Sales Example

Big Data - LAB1 Hadoop

February 4, 2026

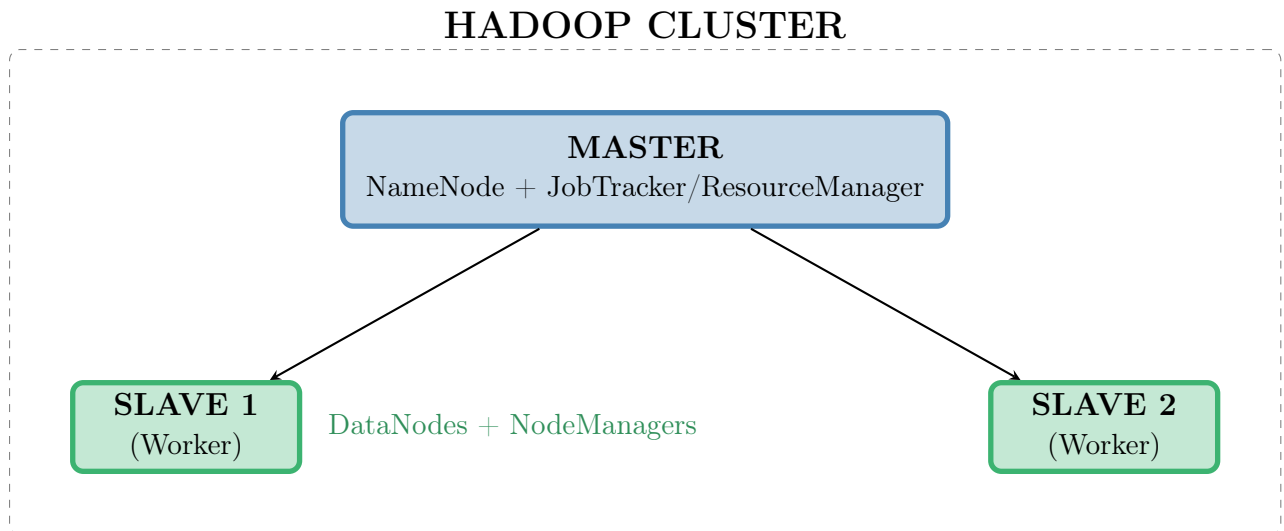
Contents

1	Hadoop Infrastructure Explained with Your Sales Example	2
2	Your Setup	2
3	HDFS (Hadoop Distributed File System)	2
3.1	In Your Example	2
4	NameNode (Lives on Master)	3
4.1	In Your Example	3
5	DataNode (Lives on Slaves/Workers)	3
5.1	In Your Example	3
6	Secondary NameNode	4
6.1	In Your Example	4
7	Master vs NameNode / Slaves vs DataNodes	4
7.1	Are they the same?	4
7.2	Visual Representation	5
8	How Your MapReduce Job Runs	5
8.1	Step-by-Step with Your Sales Example	5
9	Summary Table	6
10	Key Insight: Data Locality	7

1 Hadoop Infrastructure Explained with Your Sales Example

This document explains each component of Hadoop and its role in processing sales data using MapReduce.

2 Your Setup



3 HDFS (Hadoop Distributed File System)

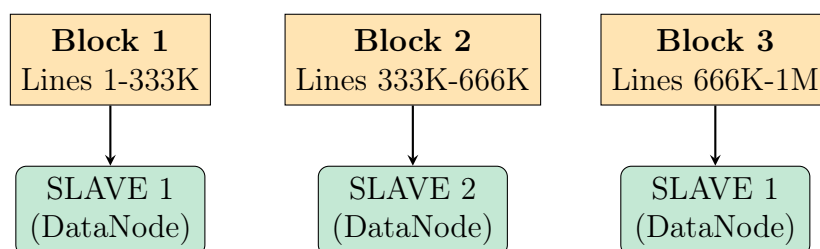
What is HDFS?

A file system that stores data **across multiple machines**.

3.1 In Your Example

When you upload `Command-sales.txt` to HDFS, here's what happens:

Command-sales.txt (1 million lines)



Key Point

HDFS splits your file into blocks (default 128MB) and distributes them!

4 NameNode (Lives on Master)

Aspect	Description
What is it?	The “brain” of HDFS - stores metadata only
What it stores	File names, directory structure, which block is on which DataNode
Does NOT store	The actual data!

4.1 In Your Example

NameNode's Memory

```
File: /user/hadoop/sales/Command-sales.txt
- Block 1 → Slave1, Slave2 (replicated)
- Block 2 → Slave2, Slave1 (replicated)
- Block 3 → Slave1, Slave2 (replicated)
```

Analogy

It's like a phonebook: It knows WHERE everything is, but doesn't have the data itself.

5 DataNode (Lives on Slaves/Workers)

Aspect	Description
What is it?	The “muscles” - stores actual data blocks
What it does	Stores blocks, sends data when requested, reports health to NameNode

5.1 In Your Example

Slave 1 (DataNode):

Block 1: Miami...
Block 3: Boston...

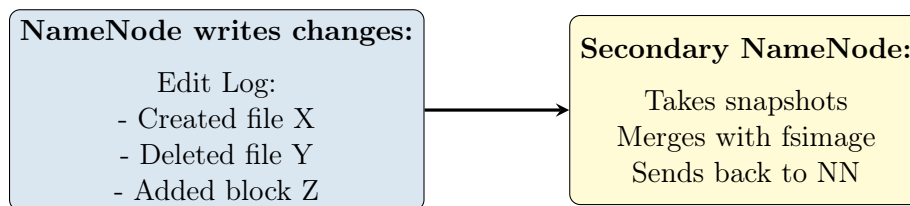
Slave 2 (DataNode):

Block 1: Miami... (replica)
Block 2: Denver...

Aspect	Description
What is it?	A helper for the NameNode (NOT a backup!)
What it does	Periodically merges edit logs with the filesystem image
Purpose	Reduces NameNode restart time

6 Secondary NameNode

6.1 In Your Example



Common Misconception

The Secondary NameNode is **NOT** a failover for NameNode!

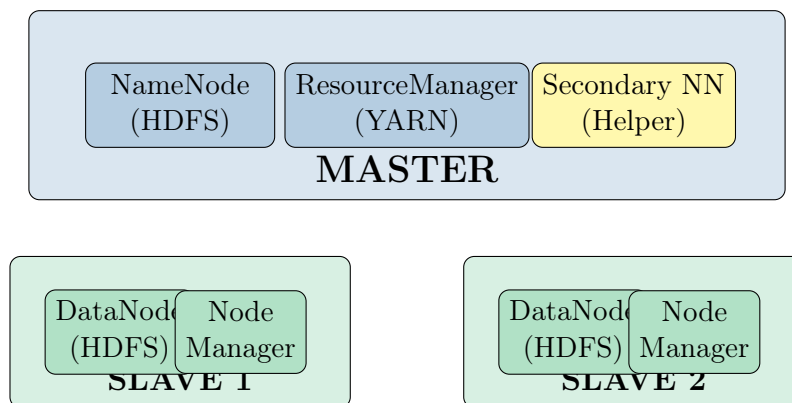
7 Master vs NameNode / Slaves vs DataNodes

7.1 Are they the same?

Not exactly! The Master and Slaves are **machines**, while NameNode/DataNode are **processes**:

Machine	Processes Running
Master	NameNode + ResourceManager (YARN) + Secondary NameNode
Slave 1	DataNode + NodeManager
Slave 2	DataNode + NodeManager

7.2 Visual Representation



8 How Your MapReduce Job Runs

8.1 Step-by-Step with Your Sales Example

Command to Run

```
hadoop jar hadoop-streaming.jar \  
  -mapper mapper.py \  
  -reducer reducer.py \  
  -input /sales \  
  -output /result
```

STEP 1: ResourceManager (Master) receives job

ResourceManager receives the job request:
"I need to process /sales/Command-sales.txt"

STEP 2: NameNode provides block locations

NameNode says:
"File has 2 blocks: Block1 → Slave1, Block2 → Slave2"

STEP 3: ResourceManager launches Mappers WHERE DATA IS

- Slave 1: Run `mapper.py` on Block 1
- Slave 2: Run `mapper.py` on Block 2

(Data locality! Don't move data, move code!)

STEP 4: Mappers output (Store, Cost) pairs

Slave 1 outputs:		Slave 2 outputs:	
Miami	999.99	Boston	79.99
Denver	150.00	Miami	29.99

STEP 5: SHUFFLE & SORT (Hadoop magic!)

Group all same keys together, sort alphabetically:

Boston: 79.99
Denver: 150.00
Miami: 999.99, 29.99

STEP 6: Reducer runs (on one or more nodes)

`reducer.py` processes sorted stream:

- `→ Boston 79.99 → print "Boston 79.99"`
- `→ Denver 150.00 → print "Denver 150.00"`
- `→ Miami 999.99 }`
- `→ Miami 29.99 } → print "Miami 1029.98"`

STEP 7: Output written to HDFS: `/result/part-00000`

Boston 79.99
Denver 150.00
Miami 1029.98

9 Summary Table

Component	Location	Role in Your Example
HDFS	Across all nodes	Stores <code>Command-sales.txt</code> split into blocks
NameNode	Master	Knows which blocks are on which slave
DataNode	Slaves 1 & 2	Actually stores the data blocks
Secondary NameNode	Master (or separate)	Helps NameNode with checkpoints
ResourceManager	Master	Schedules where mappers/reducers run
NodeManager	Slaves 1 & 2	Runs <code>mapper.py</code> and <code>reducer.py</code>

10 Key Insight: Data Locality

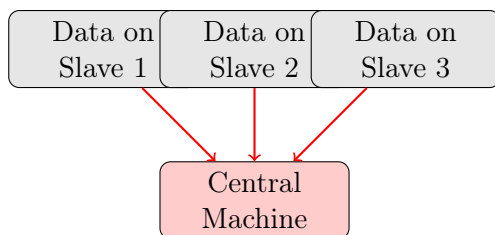
“Move computation to data, not data to computation”

Approach

55 Copy all data to one machine → Process → **Slow!**

51 Send `mapper.py` (tiny file) to where data already is → **Fast!**

Wrong Approach:



Hadoop Approach:

